Automotive Workshop API

Tech Challenge — Fase 2 · Pós-Tech FIAP Software Architecture

Links da entrega

Repositório GitHub	https://github.com/Os-Fiapudos/automotive-workshop-api Compartilhado com o usuário `soat-architecture`
Vídeo demonstrativo	https://youtu.be/Cw9C57M0s0I YouTube, não listado, 15 minutos: deploy, execução do CI/CD, consumo das APIs e escalabilidade automática
Documentação	README do repositório, com instruções de execução local, deploy em Kubernetes, provisionamento com Terraform e as collections da API (Postman, Insomnia, Bruno e OpenAPI)

Integrantes

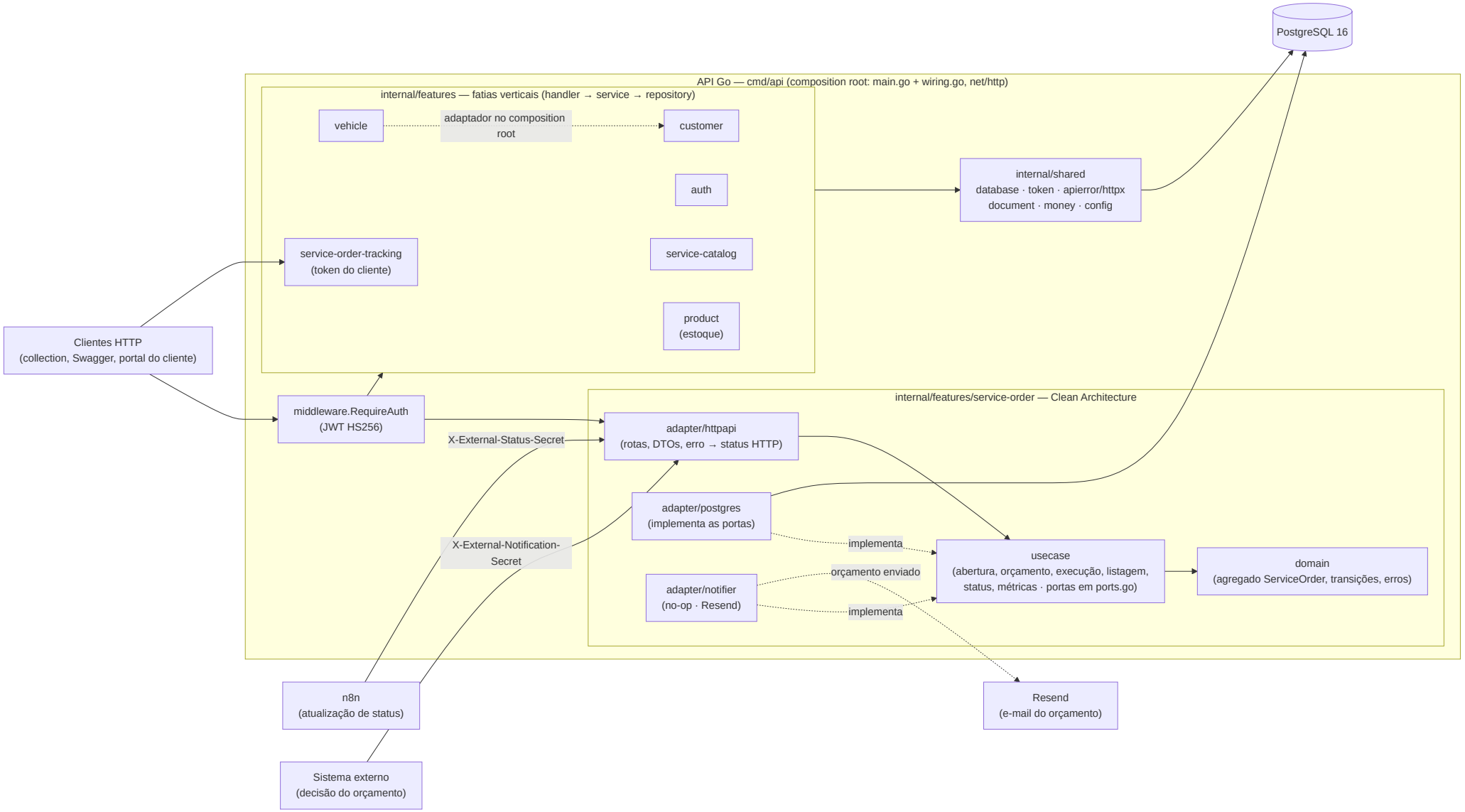
Nome	RM
Jean Ferreira dos Santos Cruz Junior	RM376169
João Victor dos Santos Cerqueira	RM376742
Giovane Kenuy Soares Colognesi Rubira Cardona	RM377047
Augusto José Rizzi	RM376119
Lucas Guimarães Fabris	RM376444

Recursos escolhidos

Aplicação	API REST em Go 1.26 (`net/http`), monólito modular em fatias verticais; `service-order` em Clean Architecture (`domain` ← `usecase` ← `adapter`), com a regra de dependência verificada por teste
Banco de dados	PostgreSQL 16 em StatefulSet com PVC de 1 Gi; schema versionado por migrations (`golang-migrate` v4.20.1) aplicadas por um Job antes de cada rollout
Contêineres	Dockerfile multi-stage (Alpine 3.24, usuário não-root, HEALTHCHECK) e docker-compose para desenvolvimento local
Orquestração	Kubernetes local com `kind` (1 nó, Kubernetes 1.37): Deployment (2 a 8 pods), Service ClusterIP, ConfigMaps, Secrets, HPA `autoscaling/v2` por CPU (70 %) e memória (80 %), metrics-server
Infraestrutura como código	Terraform 1.16 (providers `tehcyx/kind` , `hashicorp/kubernetes` , `hashicorp/helm`): cluster, namespace, metrics-server e PostgreSQL
CI/CD	GitHub Actions: build, vet, testes, cobertura ≥ 80 %, verificação de migrations, segurança (govulncheck, gosec), imagem publicada no GHCR e deploy no cluster com smoke autenticado
Integrações	n8n (atualização de status da OS por ferramenta externa) e Resend (e-mail de orçamento ao cliente)

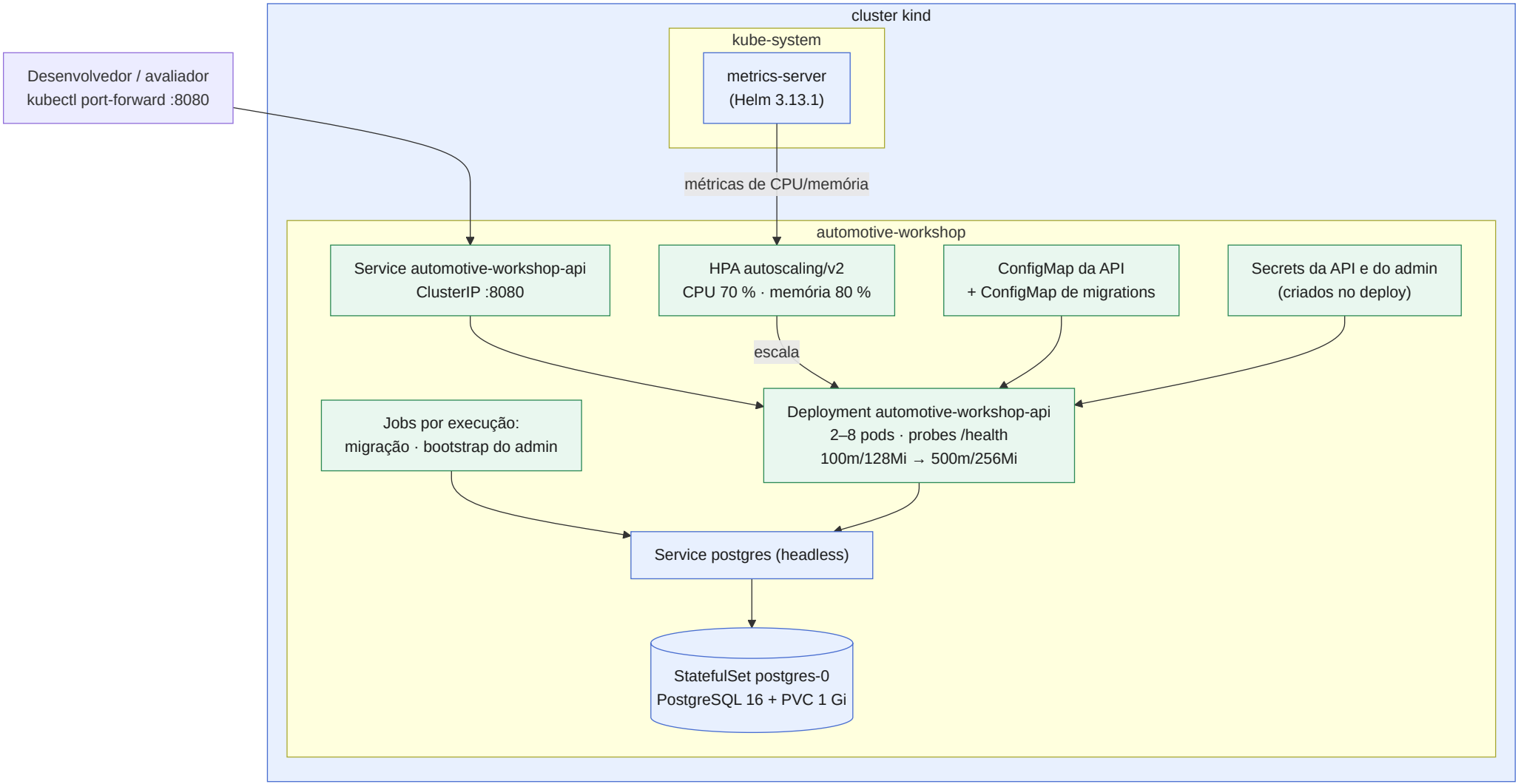
Desenho da arquitetura — Componentes da aplicação

Fatias verticais independentes; o service-order é organizado internamente em camadas de Clean Architecture.



Desenho da arquitetura — Infraestrutura provisionada

Azul: criado pelo Terraform (/infra). Verde: criado pelo deploy a partir de /k8s.



Desenho da arquitetura — Fluxo de deploy

Checks em todo push e PR; publicação e deploy no cluster em push na main ou execução manual.

